

# SPANS-99: Best Practice Summary

**Series:** SPANS | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 03/25/2026

Definitive best practice settings for distributed tracing and span analysis. Each entry specifies the exact configuration.

## Table of Contents

- 1. [DQL Syntax Rules](#)
- 2. [Filtering & Performance](#)
- 3. [Time Range & Cost](#)
- 4. [Duration & Aggregation](#)
- 5. [Trace Analysis](#)
- 6. [Service Dependencies](#)
- 7. [Security](#)
- 8. [OpenPipeline Span Configuration](#)
- 9. [Bucket Strategy & Sampling](#)

## 1. DQL Syntax Rules

| Practice          | Recommended Setting/Value                                                                   | Priority |
|-------------------|---------------------------------------------------------------------------------------------|----------|
| Equality operator | <code>==</code> (double equals) — never <code>=</code>                                      | Critical |
| Array literals    | <code>{"a", "b"}</code> with curly braces and double quotes — never <code>('a', 'b')</code> | Critical |

| Practice                             | Recommended Setting/Value                                                                                                                   | Priority |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|----------|
| NULL checks                          | <code>isNull(field) / isNotNull(field)</code> — never <code>== null</code>                                                                  | Critical |
| Multi-value matching                 | <code>in(field, {"val1", "val2"})</code> — never SQL <code>IN</code>                                                                        | Critical |
| <code>span.kind</code> values        | Lowercase only: <code>"server"</code> , <code>"client"</code> , <code>"internal"</code> , <code>"producer"</code> , <code>"consumer"</code> | Critical |
| <code>span.status_code</code> values | Lowercase only: <code>"error"</code> , <code>"ok"</code> , <code>"unset"</code>                                                             | Critical |
| String literals                      | Double quotes everywhere — never single quotes                                                                                              | Critical |
| Grouping syntax                      | <code>summarize count(), by:{field}</code> with curly braces — never <code>GROUP BY</code>                                                  | Critical |
| Always alias aggregation results     | <code>summarize error_count = count()</code> — required for <code>sort</code> and <code>fieldsAdd</code>                                    | Critical |

## 2. Filtering & Performance

| Practice                                                             | Recommended Setting/Value                                                                                                                                                                 | Priority    |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| Filter immediately after fetch                                       | All <code>filter</code> commands before any <code>fieldsAdd</code> , <code>summarize</code> , <code>sort</code>                                                                           | Critical    |
| Filter on indexed fields first                                       | <code>trace.id</code> , <code>span.id</code> , <code>dt.entity.service</code> , <code>span.name</code> , <code>span.kind</code> , <code>span.status_code</code> , <code>start_time</code> | Critical    |
| Use <code>dt.entity.service</code> over <code>service.name</code>    | Always populated and indexed vs. sometimes missing and not indexed                                                                                                                        | Recommended |
| <code>==</code> for exact matches, <code>~</code> for wildcards only | <code>field == "exact"</code> is faster than <code>field ~ "exact"</code>                                                                                                                 | Recommended |
| Combine conditions in single filter                                  | <code>filter span.kind == "server" and isNotNull(dt.entity.service) and duration &gt; 100ms</code>                                                                                        | Recommended |

| Practice                                | Recommended Setting/Value                                                                                              | Priority    |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------|-------------|
| Most restrictive filters first          | Order from most selective to least selective                                                                           | Recommended |
| Select only needed fields               | <code>fields start_time,</code><br><code>dt.entity.service, span.name,</code><br><code>duration</code> after filtering | Recommended |
| Compute derived fields after filtering  | <code>fieldsAdd duration_ms = ...</code> must appear after all <code>filter</code> steps                               | Recommended |
| Sort after summarize, never after fetch | Sorting before aggregation wastes resources                                                                            | Critical    |
| Limit after sort, at pipeline end       | <code>sort desc \   limit 20</code> as final commands — never <code>limit</code> before <code>summarize</code>         | Critical    |

### 3. Time Range & Cost

| Practice                                                   | Recommended Setting/Value                                                                                                           | Priority    |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-------------|
| Always specify explicit time range                         | <code>fetch spans, from:-1h</code> — never bare <code>fetch spans</code>                                                            | Critical    |
| Use narrowest range possible                               | Debugging: <code>from:-5m</code> . Recent: <code>from:-1h</code> .<br>Daily: <code>from:-24h</code> . Weekly: <code>from:-7d</code> | Critical    |
| Target specific buckets                                    | <code>fetch spans, bucket:{"default_spans"}</code><br>— avoid scanning all buckets                                                  | Recommended |
| Never group by <code>trace.id</code> without pre-filtering | Millions of unique values — filter by error or time first                                                                           | Critical    |
| Group by low-cardinality fields for dashboards             | <code>dt.entity.service, span.name,</code><br><code>span.kind, http.route</code> — never <code>url.path, trace.id</code>            | Recommended |
| Always <code>limit</code> non-aggregated queries           | <code>limit 100</code> on any raw span query                                                                                        | Critical    |

| Practice                    | Recommended Setting/Value                               | Priority    |
|-----------------------------|---------------------------------------------------------|-------------|
| Use only needed percentiles | p50 + p95 for monitoring; add p99 only for SLO tracking | Recommended |

## 4. Duration & Aggregation

| Practice                                             | Recommended Setting/Value                                                                                                                                                                                                  | Priority    |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| Convert duration from nanoseconds                    | <code>/1000000.0</code> for ms, <code>/1000000000.0</code> for seconds                                                                                                                                                     | Critical    |
| Use duration literals in filters                     | <code>filter duration &gt; 100ms</code> , <code>filter duration &gt; 1s</code>                                                                                                                                             | Recommended |
| Use <code>countIf()</code> for conditional counts    | <code>summarize errors = countIf(span.status_code == "error"), total = count()</code> in single pass                                                                                                                       | Recommended |
| Error rate calculation                               | <code>(error_count * 100.0) / total_requests</code> — use <code>100.0</code> (float) to avoid integer division                                                                                                             | Recommended |
| Percentile trends with <code>bin()</code>            | <code>fieldsAdd time_bucket = bin(start_time, 10m) \</code><br><code>  summarize p95 = percentile(duration, 95)/1000000, by: {time_bucket}</code> — <code>makeTimeseries</code> does NOT support <code>percentile()</code> | Critical    |
| Use <code>makeTimeseries</code> for dashboard charts | <code>makeTimeseries requests = count(), errors = countIf(span.status_code == "error"), interval:5m</code>                                                                                                                 | Recommended |
| Minimum sample size before ranking                   | <code>filter request_count &gt; 10</code> before percentile/error rate rankings                                                                                                                                            | Recommended |

## 5. Trace Analysis

| Practice        | Recommended Setting/Value                  | Priority    |
|-----------------|--------------------------------------------|-------------|
| Find root spans | <code>filter isNull(span.parent_id)</code> | Recommended |

| Practice                          | Recommended Setting/Value                                                                                                             | Priority    |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-------------|
| Reconstruct trace chronologically | <code>filter trace.id == "ID" \   sort start_time asc</code>                                                                          | Recommended |
| Find first error in a trace       | <code>summarize first_error_time = min(start_time), service = takeFirst(service.name), by:{trace.id}</code><br>after filtering errors | Recommended |
| Detect cascading failures         | <code>filter span.status_code == "error" \   summarize count(), by:{trace.id} \   filter count &gt; 1</code>                          | Recommended |
| Impact score for bottleneck spans | <code>fieldsAdd impact_score = call_count * avg_duration_ms \   sort impact_score desc</code>                                         | Recommended |
| Health scorecard classification   | <code>if(error_rate &gt; 5, "Critical", else: if(error_rate &gt; 1, "Warning", else: "Healthy"))</code>                               | Recommended |

## 6. Service Dependencies

| Practice                                             | Recommended Setting/Value                                                                                                                                                | Priority    |
|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| Map dependencies via CLIENT spans                    | <code>filter span.kind == "client" \   summarize count(), by:{service.name, server.address}</code>                                                                       | Recommended |
| Use <code>peer.service</code> for named dependencies | <code>filter span.kind == "client" and isNotNull(peer.service) when available</code>                                                                                     | Recommended |
| Track async messaging                                | <code>filter span.kind in {"producer", "consumer"} \   summarize count(), by:{messaging.system, messaging.destination.name}</code>                                       | Recommended |
| Outbound/inbound ratio                               | <code>summarize inbound = countIf(span.kind == "server"), outbound = countIf(span.kind == "client"), by:{service.name}</code> — high ratio = heavy downstream dependency | Optional    |

## 7. Security

| Practice                                                               | Recommended Setting/Value                                                                                                                   | Priority    |
|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| Use <code>http.route</code> in dashboards, never <code>url.path</code> | <code>http.route</code> = pattern (no PII), <code>url.path</code> = actual values (may contain PII)                                         | Critical    |
| Weekly PII audit: emails in URLs                                       | <code>filter contains(url.path, "@")</code> or <code>contains(url.path, "%40")</code> — must return 0                                       | Critical    |
| Weekly audit: tokens in URLs                                           | <code>filter contains(url.path, "password")</code> or <code>contains(url.path, "token")</code> or <code>contains(url.path, "secret")</code> | Critical    |
| Weekly audit: sensitive data in <code>db.statement</code>              | <code>filter contains(db.statement, "password")</code> or <code>contains(db.statement, "ssn")</code>                                        | Critical    |
| Monitor 401/403/429 as security signals                                | <code>filter in(http.response.status_code, {401, 403, 429})</code> with 10-min bucketing on dashboard                                       | Critical    |
| Brute force detection                                                  | 401 count >10 per hour per endpoint = alert                                                                                                 | Critical    |
| Enumeration detection                                                  | 404 count >100 with high unique path count = scanning alert                                                                                 | Recommended |
| Limit <code>trace.id</code> exposure                                   | <code>takeFirst(trace.id)</code> per error pattern — never expose all trace IDs                                                             | Recommended |
| Mask emails in span URLs via OpenPipeline                              | Replace <code>[a-zA-Z0-9._%+-]+@...</code> with <code>***@***.***</code> on <code>http.url</code>                                           | Critical    |
| Mask tokens in query params via OpenPipeline                           | Replace <code>(token\ key\ secret\ password)=([^&amp;]+)</code> with <code>\$1=***REDACTED***</code>                                        | Critical    |
| Monthly full PII audit                                                 | Check: emails, tokens, user IDs in URLs + sensitive data in <code>db.statement</code> + <code>exception.stacktrace</code>                   | Critical    |

## 8. OpenPipeline Span Configuration

| Practice                                            | Recommended Setting/Value                                                                                                       | Priority    |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|-------------|
| Drop health check spans at ingestion                | <code>contains(span.name, "health")</code> or <code>contains(span.name, "ready")</code>                                         | Critical    |
| Drop static asset spans                             | <code>endsWith(span.name, ".js")</code> or <code>endsWith(span.name, ".css")</code> or <code>endsWith(span.name, ".png")</code> | Recommended |
| Never drop error spans                              | <code>span.status_code == "error"</code> → keep at 100% regardless of sampling                                                  | Critical    |
| Never drop slow spans                               | <code>duration &gt; 1s</code> → keep at 100% regardless of sampling                                                             | Critical    |
| Pre-compute <code>duration_ms</code> at ingestion   | Transform: <code>duration_ms = duration / 1000000</code>                                                                        | Optional    |
| Route to environment buckets                        | Production: <code>spans_production_90d</code> . Staging: <code>spans_staging_7d</code> . Dev: <code>spans_default_3d</code>     | Recommended |
| Route sensitive service spans to restricted buckets | Payment/auth services → <code>spans_sensitive</code> with restricted IAM                                                        | Recommended |
| Bucket-level IAM for team isolation                 | Frontend team: <code>spans_frontend</code> only. SRE: all span buckets. Compliance: <code>audit_spans</code>                    | Recommended |

## 9. Bucket Strategy & Sampling

| Practice                 | Recommended Setting/Value                                            | Priority    |
|--------------------------|----------------------------------------------------------------------|-------------|
| Retention by environment | Production: 90 days. Standard: 35 days. Staging: 7 days. Dev: 3 days | Recommended |
| Three-tier sampling      | (1) Errors → 100%. (2) Slow >1s → 100%. (3) Everything else → 10-50% | Critical    |

| Practice                                      | Recommended Setting/Value                                                                            | Priority    |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------|-------------|
| Sample high-volume services at 10%            | Services generating >10,000 spans/hour                                                               | Recommended |
| Calculate savings before filtering            | Count health checks + static + errors to quantify droppable vs must-keep                             | Recommended |
| Extract <code>dt.ingest.size</code> as metric | Key: <code>span.ingest.size.by.app</code> , value: <code>dt.ingest.size</code> , dim: app identifier | Recommended |
| Verify bucket distribution monthly            | <code>summarize count(), by: {dt.system.bucket}</code> — verify routing rules working                | Recommended |

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*